PhotoMatch — Manual Técnico de Defensa

Trabajo Práctico 07: Contenedores, Integración y Pruebas E2E

Ingeniería de Software III · Universidad Católica de Córdoba (UCC) · 2026

Alumno: Lautaro Alejo Vallino
Legajo: 2301009 · Comisión 2
Aplicación: PhotoMatch (Node.js / EJS / PostgreSQL)

Release Oficial: `v7.0.0`
Herramienta E2E: Playwright (Chromium Headless)
Gate Demostrado: Bloqueo PROD ante fallo E2E

A. Introducción Conceptual

1. Integración vs. End-to-End (E2E)

- **Pruebas de Integración (API Testing):** Verifican la comunicación real entre el servidor Express y PostgreSQL sin interfaz gráfica, utilizando el contexto HTTP directo de Playwright (`request`). Se ejecutan en milisegundos y aíslan fallos en el contrato de datos.
- **Pruebas End-to-End (E2E):** Validan el recorrido de usuario completo en un navegador Chromium real automatizado con Playwright, verificando renderizado de plantillas EJS, redirecciones HTTP y mutaciones del DOM.

2. El Mecanismo del Gate E2E Rojo

Un pipeline de Continuous Delivery robusto debe aislar la causa raíz de un defecto:

- Si la base y la API responden pero la interfaz web se rompe: la integración pasa verde pero **E2E falla en rojo**.
- Mediante la dependencia `needs: e2e` en el workflow de GitHub Actions, el despliegue a producción se **omite y bloquea automáticamente**, impidiendo que el defecto afecte a los usuarios de Producción.

B. Estado Inicial de PhotoMatch antes del TP7

- Solo se contaba con pruebas unitarias en memoria (TP5) y despliegue básico (TP6).
- No existían pruebas automatizadas contra la base de datos real ni validaciones de interfaz.
- La plantilla `form.ejs` arrastraba una colisión sintáctica con EJS (`client` vs `customer`) que provocaba HTTP 500 al renderizar.

C. Implementación Paso a Paso

1. **Diseño de Suite de Integración API (`frontend/e2e/api.spec.js`):** 3 pruebas directas: alta/lectura/baja de cliente, validación de errores HTTP 400 y actualización con verificación en base de datos.
2. **Diseño de Suite E2E de Navegador (`frontend/e2e/app.spec.js`):** 3 pruebas con Chromium: flujo completo de autenticación y alta de cliente, visualización de errores de validación, y edición persistente post-recarga.
3. **Ejecución de la Demostración del Gate E2E Rojo:**
   - **PR 37:** Rompió deliberadamente el formulario HTML apuntando el POST a `/clients/blocked-demo`.
   - **CI Run 40283556436:** `integracion` pasó 3/3 verde; `e2e` falló 3/3 en rojo con HTTP 404; `deploy-prod` quedó **bloqueado y omitido**. PROD permaneció en el SHA sano.
   - **PR 39:** Restauró la ruta a `/clients`.

- **CI Run 40284305415:** Integración y E2E pasaron 3/3 verdes. Se aprobó el gate humano y se desplegó Producción en commit `1caea7c`.

D. Decisiones de Ingeniería y Trade-offs

Decisión Tomada	Alternativa Descartada	Justificación Técnica y Trade-off
Playwright Unificado	Supertest (API) + Cypress (UI)	Playwright resuelve API y navegador en la misma biblioteca, con soporte nativo de Chrome DevTools Protocol y auto-waiting.
Pruebas contra QA Real	Servidor Express efímero en memoria	Certifica la convivencia en contenedores Docker y PostgreSQL real tal como se desplegará en Producción.
Limpieza en Cada Prueba	Truncar la base completa	Truncar la base destruye usuarios administradores semilla. Cada test genera nombres únicos y los elimina puntualmente.

E. Pruebas y Evidencias Reales

Evidencias Auditables en GitHub

1. **Gate E2E Rojo Demostrado:** PR 37 fusionado a `main`. CI Run 40283556436: Integración 3/3 verde (26s); E2E 3/3 rojo (2m 08s); `deploy-prod` bloqueado y omitido. Producción permaneció intacta en SHA `43f142e`.
2. **Artefacto de Evidencia:** Reporte Playwright con screenshots y traces subido como `playwright-report-e2e` (ID 12632244336).
3. **Restauración y Promoción Verde:** PR 39 fusionado a `main`. CI Run 40284305415: Integración y E2E 3/3 verdes; `deploy-prod` aprobado y desplegado con commit `1caea7ce1e19d4b2b84676ec8a35686f6e26f2e3`.
4. **Release Oficial:** `v7.0.0` publicada en GitHub con notas y evidencias.

F. Preparación para la Defensa Oral

Pregunta 1: ¿Por qué la suite de integración pasó en verde durante el PR 37 si el formulario estaba roto?

Respuesta para defender:

Porque el cambio defectuoso del PR 37 afectó exclusivamente el atributo `action` del formulario HTML en la plantilla EJS. La API REST en `/api/clients` y el controlador continuaron funcionando correctamente, demostrando la capacidad del pipeline de aislar fallos por capa.

Pregunta 2: ¿Cómo impidió el pipeline que el fallo E2E rompiera la producción?

Respuesta para defender:

En `.github/workflows/ci.yml`, el job `deploy-prod` declara la directiva `needs: e2e`. Al fallar el job `e2e`, GitHub Actions evalúa la dependencia insatisfecha y omite automáticamente el despliegue a producción sin solicitar aprobación.

Pregunta 3 (Difícil): ¿Cómo mitigan los flaky tests en las pruebas E2E de navegador?

Respuesta para defender:

Eliminando esperas fijas (`sleep`) y utilizando el auto-waiting de Playwright (actionability checks que esperan estabilidad del DOM). Además, configuramos reintentos automáticos (`retries: 2` en CI) y registros con identificadores temporales únicos para evitar condiciones de carrera.

G. Resumen Final de Estudio

- **Integración:** 3/3 pruebas de API REST contra PostgreSQL sin navegador.
- **E2E:** 3/3 pruebas completas con Chromium headless simulando usuarios reales.
- **Gate E2E Rojo:** Bloqueo automático de Producción ante rotura de interfaz certificado en CI Run 40283556436.
- **Pitch de 2 minutos:** *«En el TP7 implementamos pruebas de integración y E2E con Playwright contra el ambiente desplegado en QA. Demostramos el funcionamiento del gate funcional: una rotura deliberada en la interfaz gráfica pasó la integración pero rompió la suite E2E, bloqueando automáticamente la promoción a producción y protegiendo el entorno real. Tras integrar la corrección, recuperamos el 100% verde y promovimos a producción con la release v7.0.0.»*